

EXP 9: LINEAR AND POLYNOMIAL REGRESSION

NAME: SAMUEL I

REG NO:192312584

AIM:

To compare the performance of Linear Regression and Polynomial Regression algorithms using Python on a given dataset.

PROCEDURE

1. Import the required libraries and create a sample dataset.
2. Train the Linear Regression model and predict the output values.
3. Convert the input data into polynomial features.
4. Train the Polynomial Regression model and predict the output values.
5. Compare both models using the R^2 score and Mean Squared Error (MSE).

PROGRAM:

```
# Compare Linear Regression and Polynomial Regression
```

```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
from sklearn.metrics import r2_score, mean_squared_error
```

```
# Sample dataset
```

```
X = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)
```

```
y = np.array([1.5,3.8,8.2,15.5,24.5,35.8,49.2,64.5,82.0,101.5])
```

```
# Linear Regression
```

```
lr = LinearRegression()
```

```
lr.fit(X, y)
```

```
y_lr = lr.predict(X)
```

```
# Polynomial Regression (Degree = 2)
```

```
poly = PolynomialFeatures(degree=2)
```

```
X_poly = poly.fit_transform(X)
```

```
pr = LinearRegression()
```

```
pr.fit(X_poly, y)
```

```
y_pr = pr.predict(X_poly)
```

```
# Results
```

```
print("Linear Regression")
```

```
print("R2 Score:", r2_score(y, y_lr))
```

```
print("MSE:", mean_squared_error(y, y_lr))
```

```
print("\nPolynomial Regression")
```

```
print("R2 Score:", r2_score(y, y_pr))
```

```
print("MSE:", mean_squared_error(y, y_pr))
```

OUTPUT:

```
Linear Regression
R2 Score: 0.9447273748242438
MSE: 60.108896969697

Polynomial Regression
R2 Score: 0.9999685238158671
MSE: 0.03423030303030339
```

2.

PROGRAM:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score
```

```
X, y = load_diabetes(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Linear Regression
```

```
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

```
# Polynomial Regression
```

```
pr = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
pr.fit(X_train, y_train)
y_pred_pr = pr.predict(X_test)
```

```
print("Linear Regression")
print("R2:", r2_score(y_test, y_pred_lr))
print("MSE:", mean_squared_error(y_test, y_pred_lr))
```

```
print("\nPolynomial Regression")
print("R2:", r2_score(y_test, y_pred_pr))
print("MSE:", mean_squared_error(y_test, y_pred_pr))
```

OUTPUT:

```
Linear Regression
R2: 0.4526027629719195
MSE: 2900.193628493482

Polynomial Regression
R2: 0.4156399336408013
MSE: 3096.0283073442656
```

3.

PRORAM:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score
```

```
X, y = fetch_california_housing(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
lr = LinearRegression()
lr.fit(X_train, y_train)
y1 = lr.predict(X_test)
```

```
pr = make_pipeline(PolynomialFeatures(2), LinearRegression())
pr.fit(X_train, y_train)
y2 = pr.predict(X_test)
```

```
print("Linear R2:", r2_score(y_test, y1))
print("Polynomial R2:", r2_score(y_test, y2))
```

OUTPUT:

```
Linear R2: 0.5757877060324524
Polynomial R2: 0.6456819711504156
```

4.

PROGRAM:

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score

np.random.seed(42)

X = np.sort(5 * np.random.rand(300, 1), axis=0)
y = X[:,0]**2 + 3*X[:,0] + np.random.randn(300)*2

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)

pr = make_pipeline(PolynomialFeatures(2), LinearRegression())
pr.fit(X_train, y_train)

print("Linear R2:", r2_score(y_test, lr.predict(X_test)))
print("Polynomial R2:", r2_score(y_test, pr.predict(X_test)))
```

OUTPUT:

```
Linear R2: 0.9589492953566109
Polynomial R2: 0.9807067850630026
```

5.

PROGRAM:

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score

X, y = make_regression(
    n_samples=1000,
    n_features=6,
    noise=20,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)

pr = make_pipeline(PolynomialFeatures(2), LinearRegression())
pr.fit(X_train, y_train)

print("Linear R2:", r2_score(y_test, lr.predict(X_test)))
print("Polynomial R2:", r2_score(y_test, pr.predict(X_test)))
```

OUTPUT:

```
Linear R2: 0.96237287061775
Polynomial R2: 0.9613018635378311
```

RESULT:

The Logistic Regression algorithm was successfully implemented in Python. The model was trained and tested on the dataset, and its performance was evaluated using the confusion matrix and accuracy, achieving effective classification results.